ScamShield Pakistan

Project Document

A rule-based web application that helps people recognize suspicious messages, job offers, scholarships and URLs

First-Semester Data Analytics Project

Python | Object-Oriented Programming | Streamlit | Individual Project

Prepared by: Hassanat Ali Moazzam
Bachelor in Data Analytics, Government College University, Faisalabad

Contents

1. Project Overview
2. Project Requirements
3. Technology Stack
4. Project Folder Structure
5. Step-by-Step Implementation
6. Mathematical Risk Score
7. Testing the Application
8. Data Analytics Component
9. Development Roadmap
10. First Task and Next Steps

1. Project Overview

ScamShield Pakistan is a web application that helps people recognize suspicious messages, job offers, scholarship announcements, and URLs before they trust them.

The project is designed to be realistic for a first-semester student. First, a working application is built using Python and Object-Oriented Programming. Then the mathematical subjects of the semester are incorporated into its risk-scoring system. The project does not depend on expensive APIs or advanced AI.

What the finished application will do

- Accept a suspicious message or URL from the user.
- Identify warning signs such as OTP requests, urgent payment demands, and suspicious links.
- Calculate a transparent risk score from 0 to 100.
- Explain why the message was flagged and recommend safe next steps.
- Display an analysis history and basic statistics about detected warning signs.

2. Project Requirements

Functional requirements

ID	Requirement
FR-01	User can enter a message for analysis.
FR-02	User can optionally enter a URL for inspection.
FR-03	System identifies predefined suspicious indicators.
FR-04	System calculates a risk score and assigns a risk category.
FR-05	System explains the indicators that affected the score.
FR-06	System stores recent analyses locally.
FR-07	Dashboard shows analysis counts and common warning signs.

Non-functional requirements

Category	Requirement
Usability	Simple interface suitable for non-technical users.
Privacy	Analyze text locally within the application; do not send submitted messages to third-party services.
Reliability	Handle empty input, malformed URLs, and unexpected errors.
Transparency	Explain that the score is a rule-based estimate, not a verified probability that a scam has occurred.
Safety	Never automatically visit suspicious URLs, submit forms on them, or download their content.

3. Technology Stack

Technology	Purpose
Python 3	Core programming, rules, classes, and mathematical calculations.
Streamlit	Web interface, forms, results, and dashboard charts.
NumPy (optional initially)	Representing indicators as vectors and experimenting with weighted scores.
JSON and Python standard library	Local history storage, URL parsing, regular expressions, and date/time handling.

Install Python 3.11 or later, then use VS Code or another editor. Streamlit and NumPy can be installed in a virtual environment.

4. Project Folder Structure

A starter project contains the application, analyzer, history storage, tests, and setup instructions.

```
ScamShield_Pakistan/
|-- app.py
|-- requirements.txt
|-- README.md
|-- scamshield/
|   |-- __init__.py
|   |-- analyzer.py
|   `-- storage.py
|-- data/
`-- tests/
```

File / Folder	Description
app.py	The Streamlit web interface and dashboard.
scamshield/	Core application logic.
analyzer.py	Message and URL rules, risk scoring.
storage.py	Local JSON history.
__init__.py	Python package initialization.
data/	Local analysis history.
tests/	Automated tests for the analyzer.
requirements.txt	Required Python packages.
README.md	Setup and usage guide.

5. Step-by-Step Implementation

Step 1: Set up Python

Extract the starter ZIP file and open the extracted folder in VS Code. In its terminal, run these commands on Windows:

```
python -m venv .venv
.venv\Scripts\Activate.ps1
pip install -r requirements.txt
```

These commands create an isolated Python environment and install the packages needed for the application.

Step 2: Run your first working version

```
streamlit run app.py
```

Streamlit starts a local web server and opens the application in your browser. You should see the ScamShield Pakistan interface with its navigation, message input, and analysis button.

Step 3: Understand the OOP design

Open `scamshield/analyzer.py`. The main class is:

```
class ScamAnalyzer:
    def analyze(self, message="", url=""):
        # Examine warning signs
        # Calculate risk score
        # Return analysis results
        ...
```

This is an example of **encapsulation**: the class groups the rules and analysis behavior together. Understand how `analyze()` calls the URL-inspection method, collects findings, and returns a dictionary containing the score, category, and explanations.

6. Mathematical Risk Score

The starter application uses a weighted sum. Each detected warning sign contributes points. For example, suppose a message triggers three indicators:

Warning indicator	Weight
Request for OTP	28
Urgent language	15
Processing-fee demand	24
Total score	**67 / 100**

The score is calculated as:

$$S = \min(100, \sum x_i w_i) \quad \text{for } i = 1 \text{ to } n$$

Here, x_i is 1 if the indicator is detected and 0 otherwise, while w_i is its assigned weight.

Connection to your mathematics subjects

Subject	How it appears in the project
Discrete Structures	Each indicator is represented by a Boolean condition.
Linear Algebra	The detected indicators form a vector, and their weights form another vector. The score is their dot product.
Basic Mathematics-II	Addition, bounds, and threshold comparisons.
Probability Distributions	Later, analyze how frequently indicators occur in a labeled dataset and investigate their statistical relationship with confirmed scam examples.

> **Important:** The weights in this first version are illustrative, not statistically learned. The resulting score is a warning index, not a 67% probability of fraud.

7. Testing the Application

Try these examples in the message box:

Test	Message	Expected
A: OTP request	"Your account will be blocked. Send your OTP and password immediately." *Example test message; do not use a real OTP.*	High
B: Processing fee	"Pay the processing fee now to complete your application."	Medium
C: Ordinary message	"Your university timetable is available on the student portal."	Low

The expected categories assume no additional URL warning signs. Verify the actual output and adjust the rules if your test results differ.

To run the automated tests:

```
python -m unittest discover -s tests
```

8. Data Analytics Component

Once the basic version works, the next milestone is a small, documented dataset. Create a CSV file with columns such as:

Column	Meaning
message_text	Example message
contains_otp_request	1 if present, otherwise 0
contains_payment_demand	1 if present, otherwise 0
contains_urgency	1 if present, otherwise 0
verified_label	1 for a verified scam example, 0 for a verified legitimate example

Use public examples with appropriate usage rights or synthetic data, and clearly document how the labels were established. Avoid labeling messages as confirmed scams based only on your own rule system.

Then analyze indicator frequencies, compare your rule-based predictions against the labels, and calculate precision and recall. This makes the project a small data analytics study, not just a Python application.

9. Development Roadmap

Milestone	Focus
1. Working application	Starter files, interface, rule-based analysis, local history, and basic tests.
2. Improve the analyzer	Add more carefully tested warning rules, improve URL parsing, and prevent overlapping indicators from unfairly inflating scores.
3. Analytics dashboard	Add a dataset, summary charts, indicator frequency analysis, and evaluation metrics.
4. Academic submission	Prepare the report, flowchart, class diagram, mathematical explanation, test cases, limitations, and demonstration.

10. First Task and Next Steps

> **Your first task:** Download the ZIP, extract it, and run `streamlit run app.py`. Once you have tried it, report whether it runs successfully or share the error you receive. The analyzer is then improved step by step, so you understand the code well enough to explain and defend it in your semester presentation.